UNIT - V

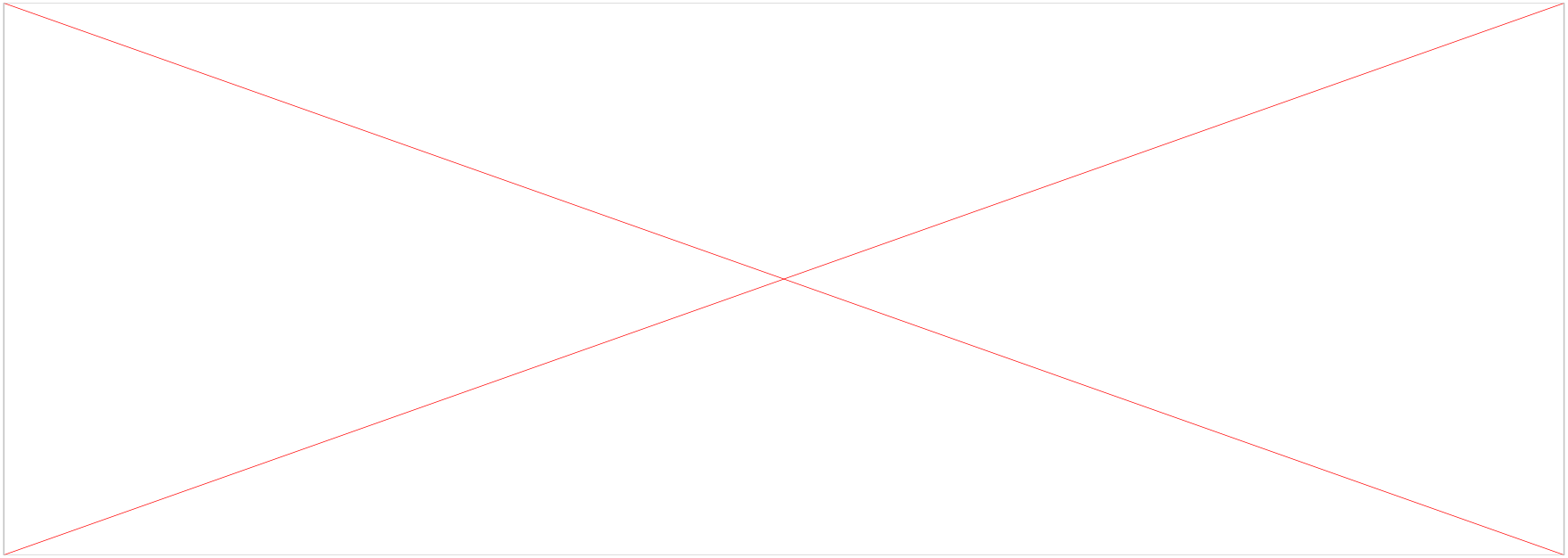
- Deep reinforcement learning
- Course Coordinator: K. Viswavardhan Reddy
- Dept of ETE, RVCE, BENGALURU

UNIT - IV

- Memory Augmented Neural Networks:
  - Neural Turing Machines (NTM)
  - Attention based memory access
  - NTM memory addressing mechanisms
  - Differentiable Neural Computers(DNC)
  - Interference-Free writing in DNCs
  - DNC memory Reuse
  - Temporal Linking of DNC Writes
  - DNC Read Head
  - DNC controller network
  - Visualizing DNC in action.

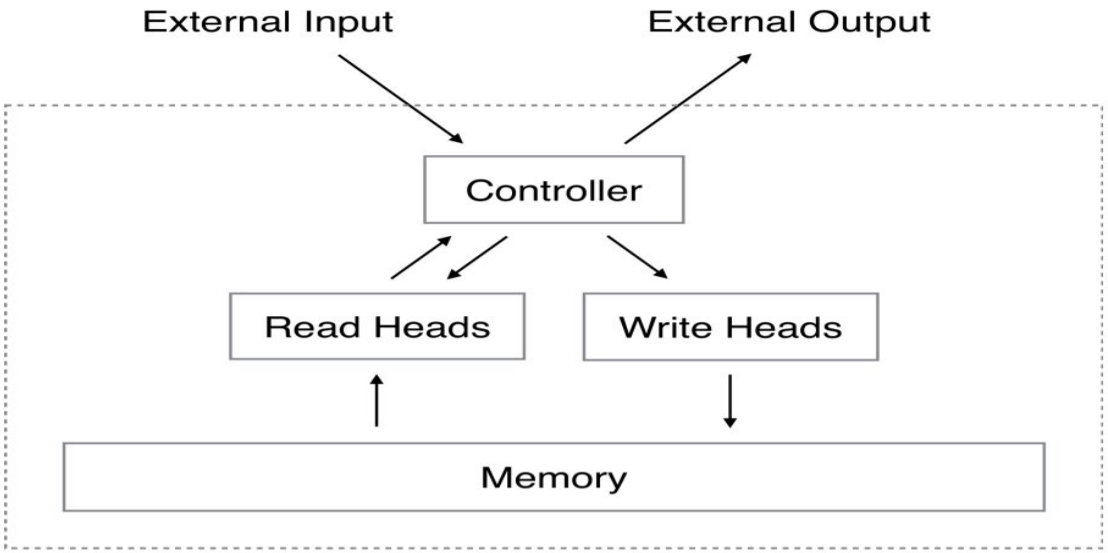
Introduction to Neural Turing Machine

- **Neural Turing Machine Architecture and Concepts**
- Neural Turing Machine is an algorithm that has the ability to store and retrieve information from memory.
  - to augment the neural network with an external memory, which used for storing and retrieving information, while usually some neural networks using hidden state as a memory.
- NTM has three core components:
  - Controller
  - Memory
  - Read and write heads



Introduction to Neural Turing Machine

- The controller, which basically a feed forward neural network or recurrent neural network, reads and writes to the memory.
- Well, the memory part, which we may call memory bank or memory matrix, here we have two-dimensional matrix, which composed of memory cell and contains N rows and M columns.
- The Controller receives input from the external environment and emits responses by interacting with the memory bank or memory matrix.
- Normally, the read head and write head are the pointers containing addresses of the memory from which it has to read from and write to.
- In NTM, we use special read and write operations to determine which location in the memory to focus on .



Neural Turing Machine Read Operations

- which one do we need to select to read from the memory? Well, it's determined by the weight vector.
- The weight vector, which can be fetched by attention mechanism, specifies which region is more import than others in memory.
- First we need to normalize the weight vector, which means that its value range from zero to one and the sum of it equals one.

Reading

Let's refer to the memory matrix, with R rows and C elements per row, at time t as $\mathcal{M}_t$. In order to read (and write), we'll need an attention mechanism that dictates where the head should read from. The attention mechanism will be a length-R normalized weight vector w_t. We'll refer to individual elements in the weight vector as $w_t(i)$. By "normalized," the authors mean that the following two constraints hold:

$$0 \leq w_t(i) \leq 1 \tag{1}$$

$$\sum_{i=1}^{R} w_t(i) = 1 \tag{1}$$

The read head will return a length-C vector r_t that is a linear combination of the memory's rows $M_t(i)$ scaled by the weight vector:

$$r_t \leftarrow \sum_{i}^{R} w_t(i) \mathcal{M}_t(i) \tag{2}$$

Writing

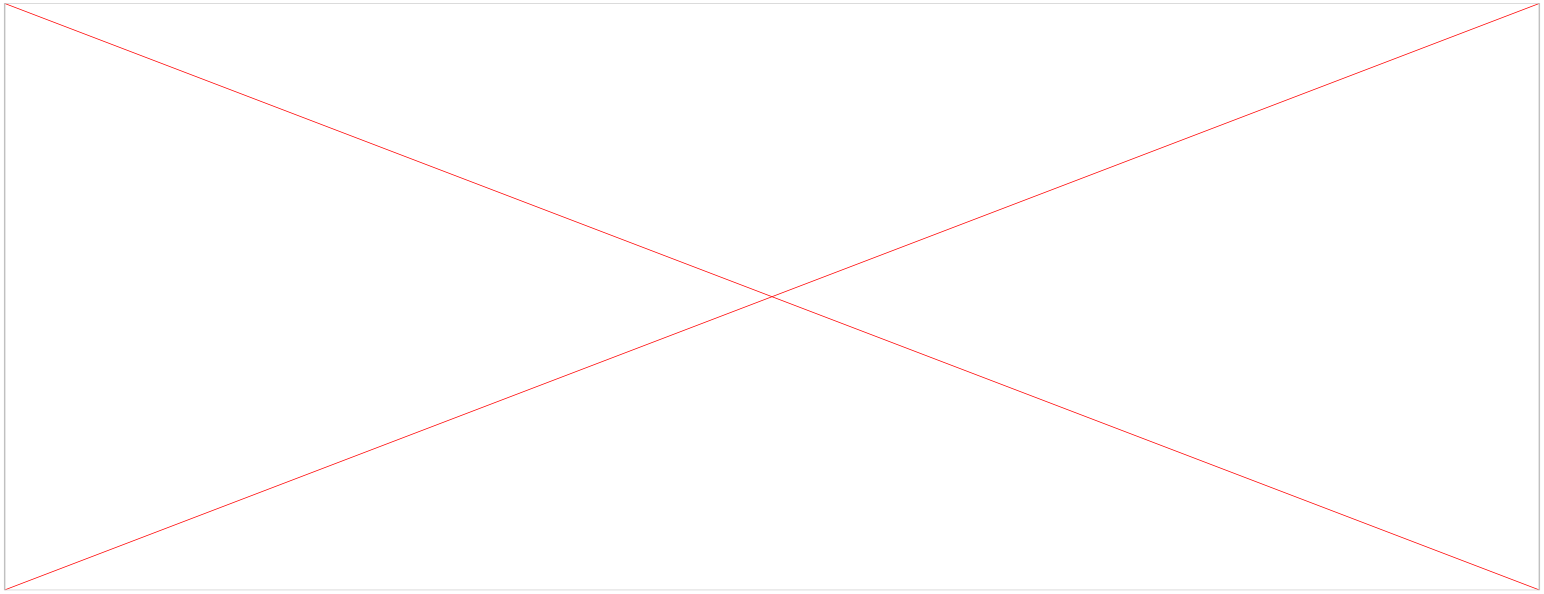
Writing is a little trickier than reading, since writing involves two separate steps: erasing, then adding. In order to erase old data, a write head will need a new vector, the length-C erase vector e_t, in addition to our length-R normalized weight vector w_t. The erase vector is used in conjunction with the weight vector to specify which elements in a row should be erased, left unchanged, or something in between. If the weight vector tells us to focus on a row, and the erase vector tells us to erase an element, the element in that row will be erased.

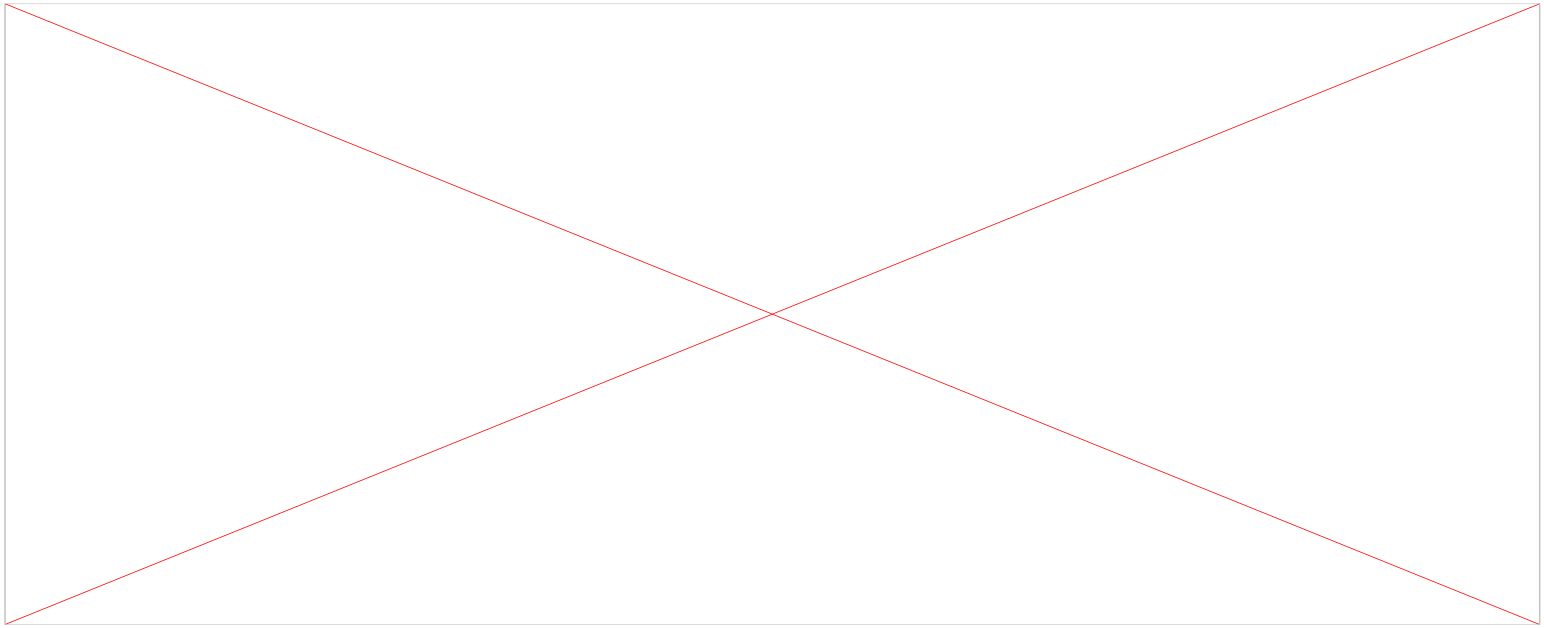
$$\mathcal{M}_t^{erased}(i) \leftarrow \mathcal{M}_{t-1}(i)[\mathbf{1} - w_t(i)e_t] \tag{3}$$

After $\mathcal{M}_{t-1}$ has been converted to $\mathcal{M}_t^{erased}$, the write head uses a length-C add vector a_t to complete the writing step.

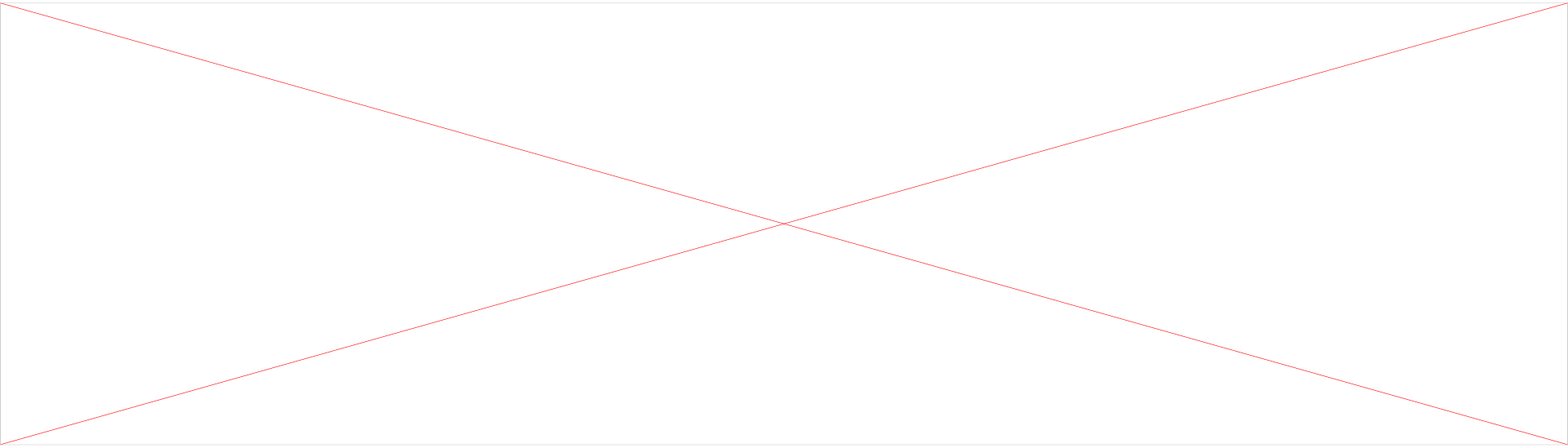
$$\mathcal{M}_t(i) \leftarrow \mathcal{M}_t^{erased}(i) + w_t(i)a_t \tag{4}$$

Neural Turing Machine Read Operations





Neural Turing Machine Read Operations



Addressing Mechanisms in Neural Turing Machine

We can use an attention mechanisms and different addressing schemes to compute the weight vector.

Here we have two types of addressing mechanisms to access information from the memory:

- Content-based addressing
- Location-based addressing

Neural Turing Machine Addressing

Content-based addressing:

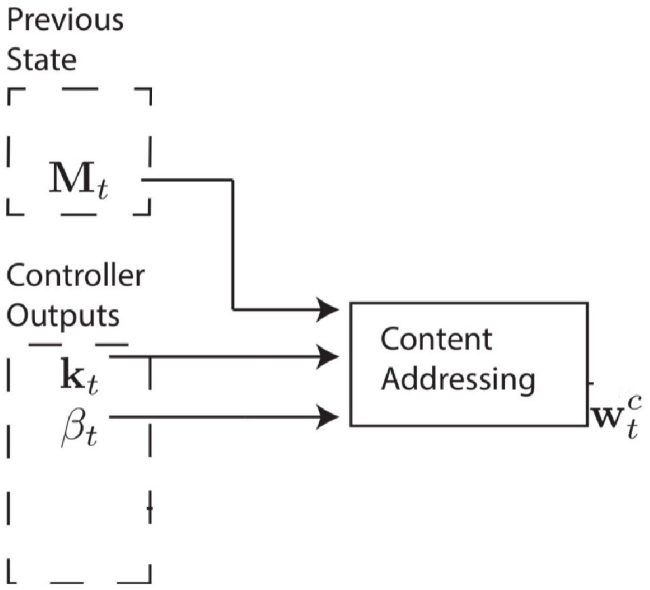
- Creating these weight vectors to determine where to read and write is tricky, so we discuss them in 4 stages:

- Each stage generates an intermediate weight vector that gets passed to the next stage.

- The first stage's goal is to generate a weight vector based on how similar each row in memory is to a length-C vector k_t emitted by the controller.

- we refer to this intermediary weight vector w_c(t) as the content weight vector.

- This content weight vector allows the controller to select values similar to previously seen values, which is called content-based addressing.



Neural Turing Machine Addressing

Content-based addressing:

For each head, the controller produces a key vector k_t that is compared to each row of M_t using a similarity measure.

$$cosine[k_t, M_t] = \frac{k_t \cdot M_t}{|k_t| \cdot |M_t|} \qquad ---(CB1)$$

The content weight vector thus is calculated as follows:

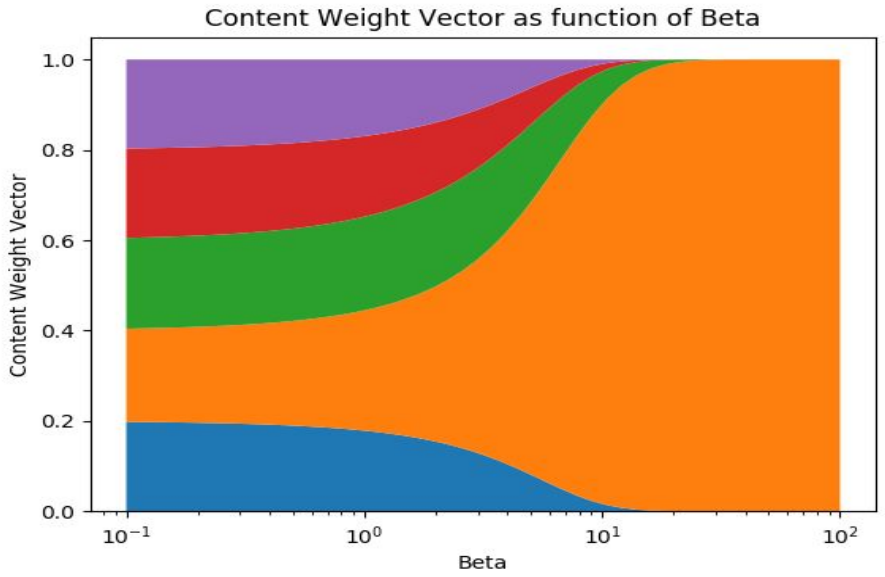
$$w_t^c = \beta_t cosine[k_t, M_t(i)] \qquad ---(CB2)$$

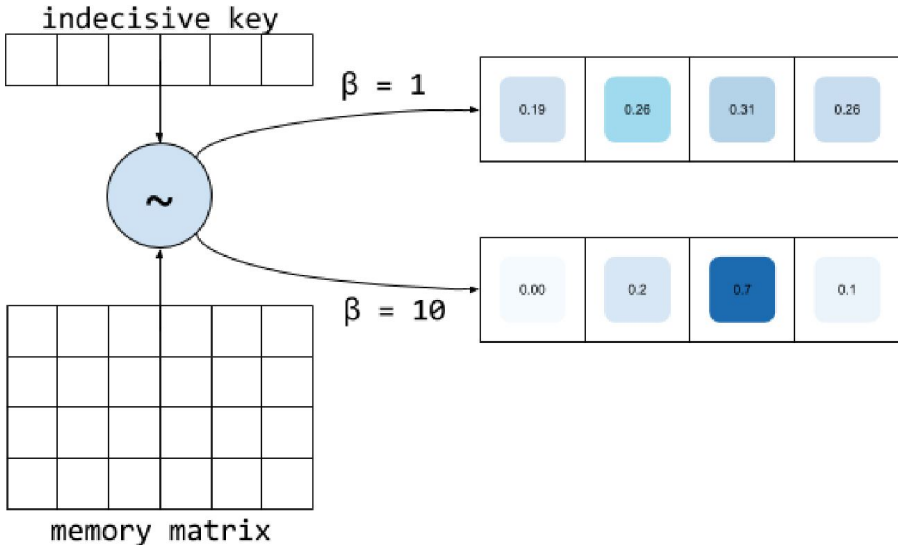
$$w_t^c = \frac{\exp(\beta_t cosine[k_t, M_t(i)])}{\sum_j \exp(\beta_t cosine[k_t, M_t(j)])} \qquad ---(CB3)$$

- A positive scalar parameter $\beta\_t > 0$, called the key strength, is used to determine how concentrated the content weight vector should be.
- For small values of beta, we focus on all the locations, but for larger values of beta, the weight vector will be concentrated on the most similar row (particular location) in memory.
- To visualize this, if a key and memory matrix produces a similarity vector [0.1, 0.5, 0.25, 0.1, 0.05], here's how the content weight vector varies as a function of beta.

Neural Turing Machine Addressing

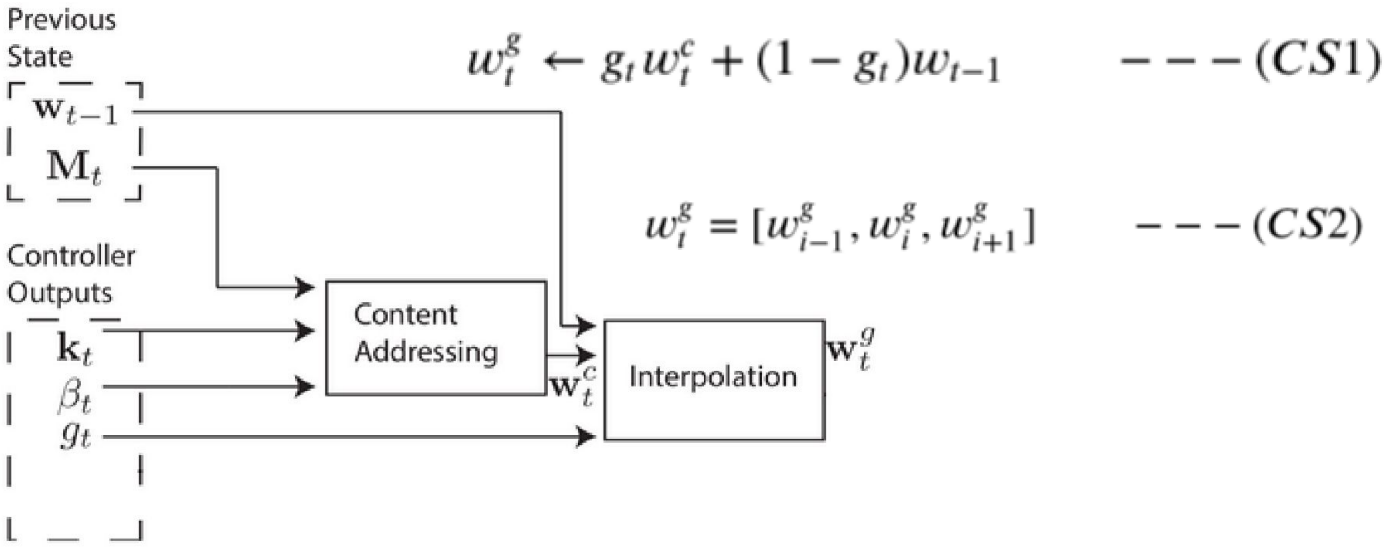
Content-based addressing:





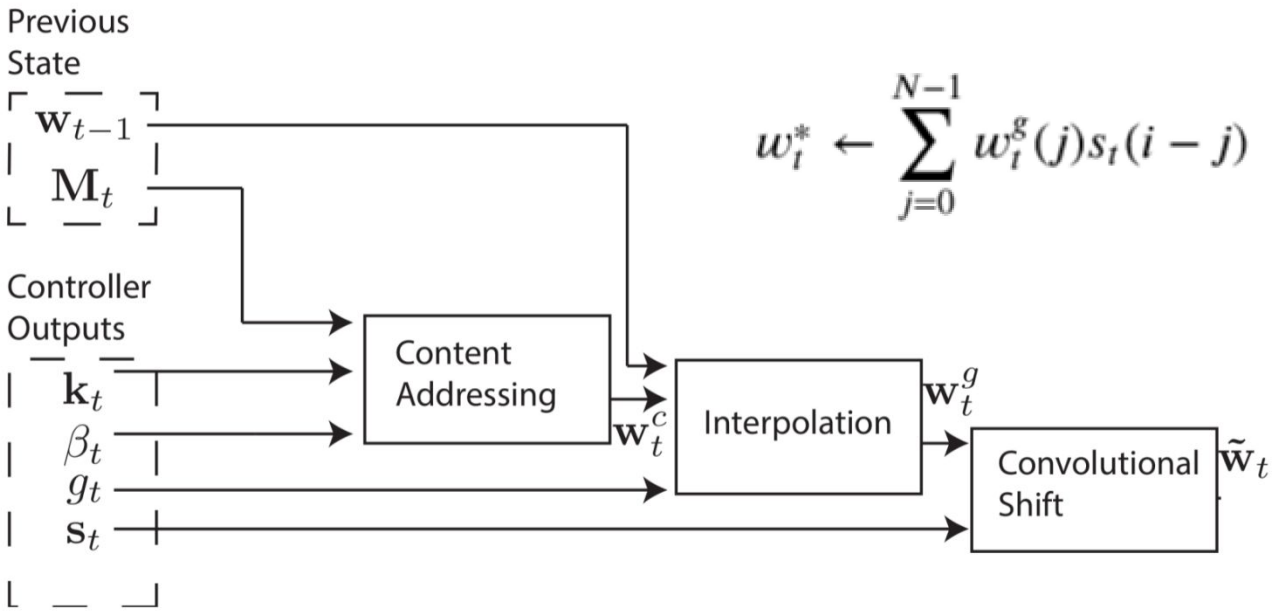
Neural Turing Machine addressing

- However, in some cases, we may want to read from specific memory locations instead of reading specific memory values.
- For that lets say we use function $f(x,y)=x*y$.
- In this case, we don't care what the values of x and y are, just that x and y are consistently read from the same memory locations.
- This is called location-based addressing, and to implement it, we'll need three more stages.
- In the second stage, a scalar parameter $g_t \in (0,1)$, called the interpolation gate, blends the content weight vector $w_c(t)$ with the previous time step's weight vector $w_t - 1$ to produce the gated weighting $w_g(t)$.
- This allows the system learn when to use (or ignore) content-based addressing.



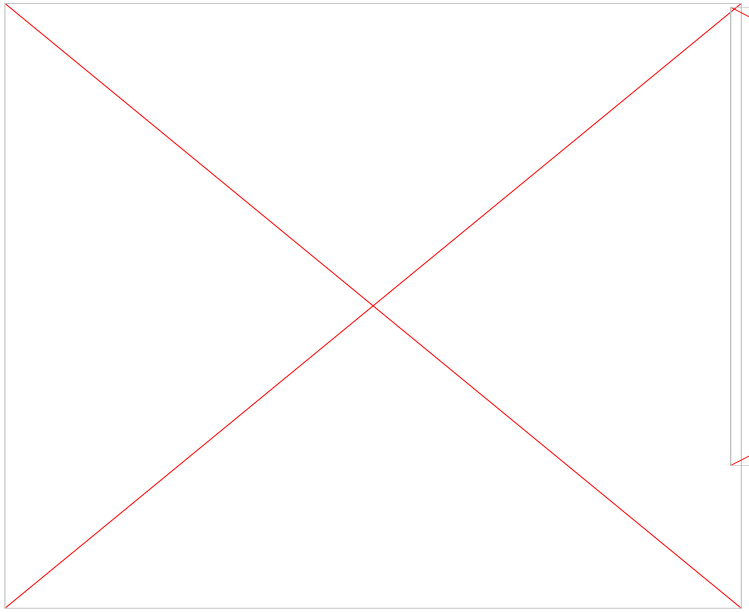
Neural Turing Machine addressing - location

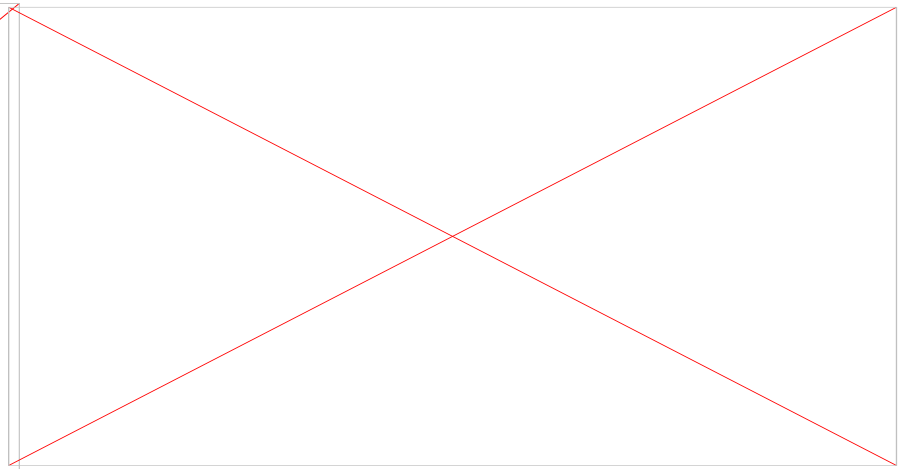
- We'd like the controller to be able to shift focus to other rows. Let's suppose that as one of the system's parameters, the range of allowable shifts is specified.
- For example, a head's attention could shift forward a row (+1), stay still (0), or shift backward a row(-1).
- We'll perform the shifts modulo *R* so that a shift forward at the bottom row of memory moves the head's attention to the top row, and similarly for a shift backward at the top row.
- After interpolation, each head emits a normalized shift weighting *s_t*, and the following convolutional shift is performed to produce the shifted weight *w~t*.

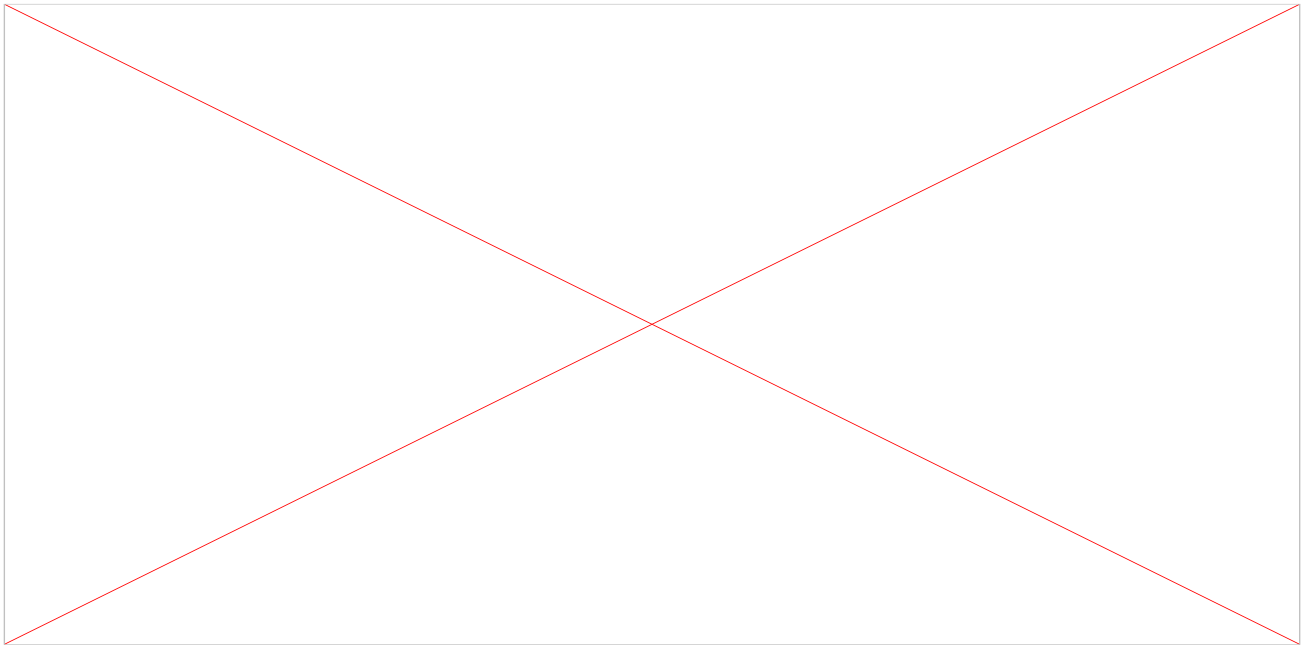


$$w_t^* \leftarrow \sum_{j=0}^{N-1} w_t^g(j) s_t(i-j) \qquad ---(CS3)$$

Neural Turing Machine addressing - Location



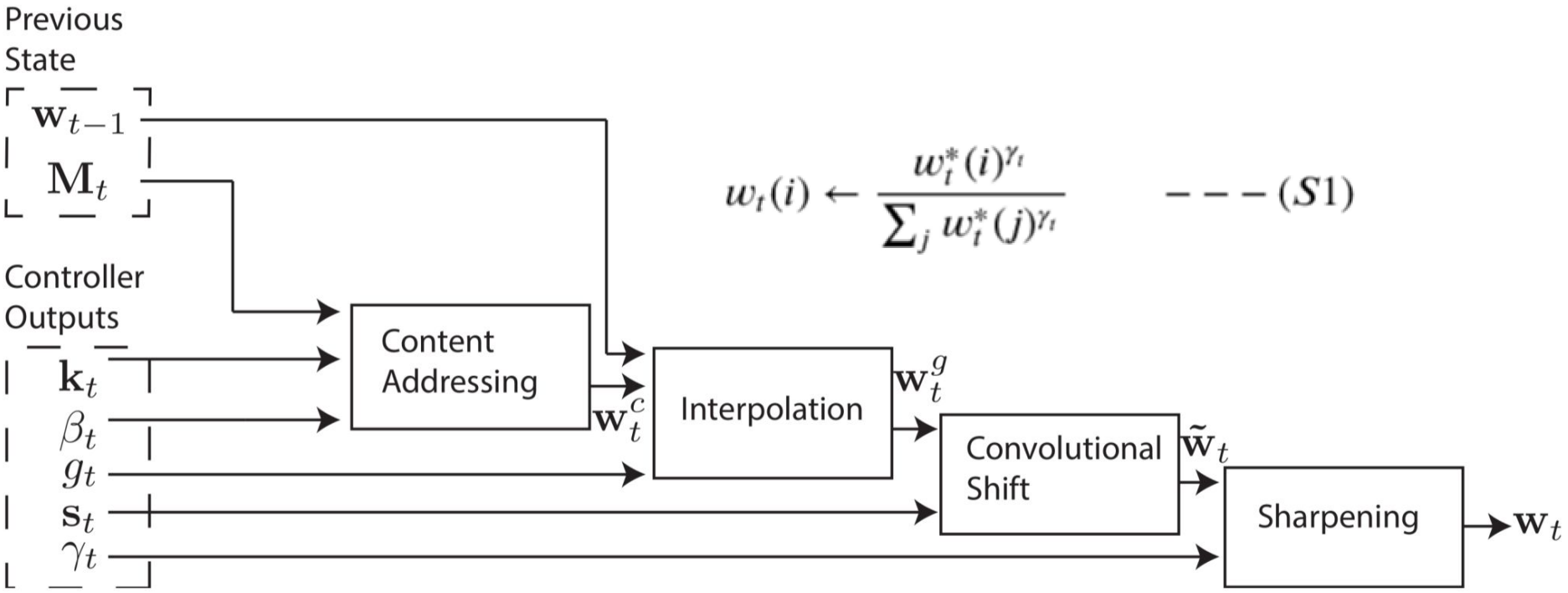




Neural Turing Machine addressing - Location

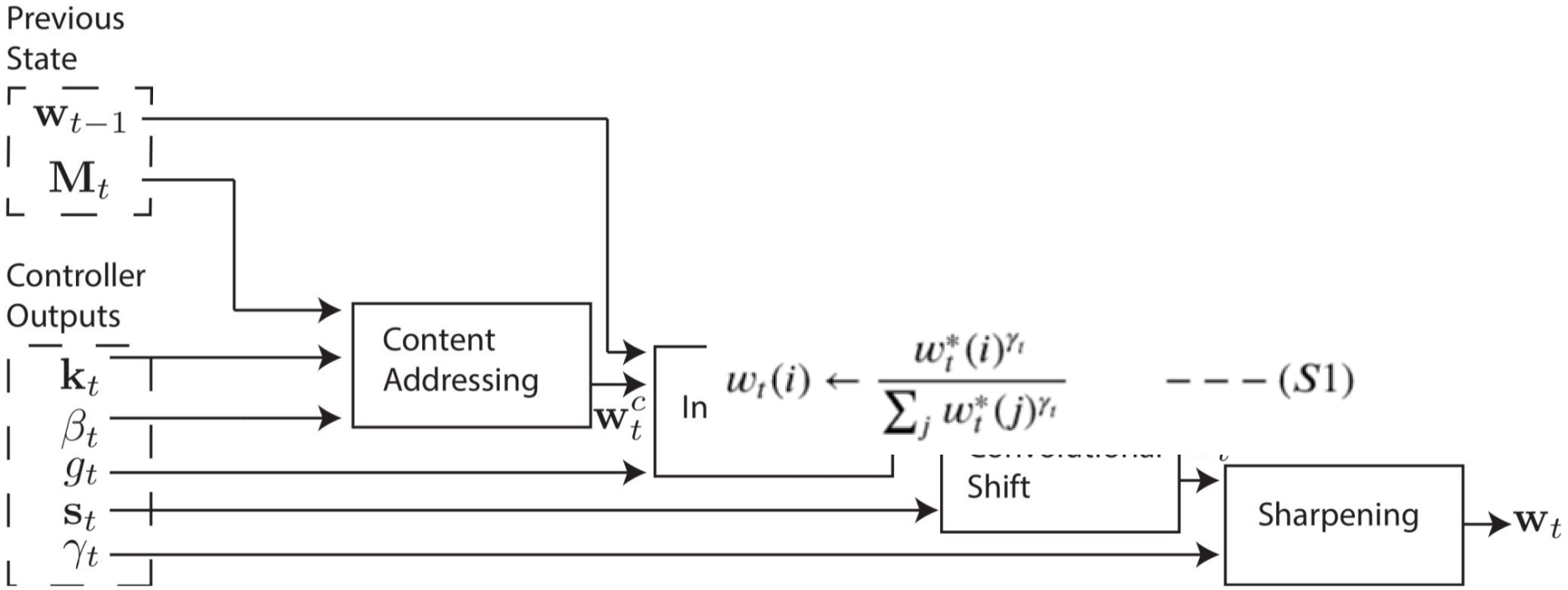
- The fourth and final stage, sharpening, is used to prevent the shifted weight *w~t* from blurring. To do this, a scalar γ_t is required.

- And that's it! A weight vector can be computed that determines where to read from and write to, and better yet, the system is entirely differentiable and thus end-to-end trainable.



Neural Turing Machine addressing - Location

- For the final step sharpening, because of the shift, weights focused at a single location will be dispersed into other locations, to mitigate this effect, sharpening was performed.



Memory-augmented Neural Network (MANN)

- one-shot learning tasks, actually is a variant of Neural Turing Machine.
- MANN can't use location-based addressing.
- MANN also use a new addressing schema called least recently used access.
- The idea behind the scene is that the least recently used memory location is determined by the read operation and the read operation is performed by content-based addressing.
- content-based addressing for reading and write to the location that was least recently used.

Read Operations On Memory-augmented Neural Network

- Read operations in MANN are the same as NTM.
- The content-based similarity can be measured by cosine similarity – similar to that of NTM.
- Unlike NTM, MANN don't use the key strength, so the softmax version of weighted vector become the third formula below.

$$cosine[k_t, M_t] = \frac{k_t \cdot M_t}{|k_t| \cdot |M_t|}$$

$$w_t^r = cosine[k_t, M_t(i)]$$

$$w_t^r = \frac{\exp(cosine[k_t, M_t(i)])}{\sum \exp(cosine[k_t, M_t(j)])}$$

$$r_t \leftarrow \sum_i^R w_t^r(i) M_t(i)$$

WRITE Operations On Memory-augmented Neural Network

- To find the least recently used memory location, we can compute a new vector called usage weight vector, say w_t^u, which will be updated after every read and write step.

- So, the usage weight vector become the sum of read weight vector and write weight vector (1st equation)

- Then we can update our usage weight vector by adding the decaying previous usage weight vector, it turns to be the formula (2nd eq)
  - (assuming the decay parameter gamma, which is used to determine how previous usage weights have to be decayed.)

$$w_t^u \leftarrow w_t^r + w_t^w$$

$$w_t^u \leftarrow \gamma w_{t-1}^u + w_t^r + w_t^w$$

$$w_t^w \leftarrow \sigma(\alpha) w_{t-1}^r + (1 - \sigma(\alpha)) w_{t-1}^{lu}$$

$$M_t(i) \leftarrow M_{t-1}^i + w_t^w(i) k_t$$

WRITE Operations On Memory-augmented Neural Network

- To compute the least recently used location, we introduce one more weight vector, say the least used weight vector w_t^{lu}.

- Compute least used weight vector from the usage weight vector is very simple.

- Simply set the index of lowest value in the usage weight vector to 1 and the rest of the values to 0, as the lowest value in the usage weight vector means that it is least recently used.

- Compute write weight vector using a sigmoid gate, which is used to compute a convex combination of the previous read weight vector and the previous least used weight vector (3rd eq).

- After computing the write weight vector, we finally update the memory matrix, that is the formula F4.

$$w_t^u \leftarrow w_t^r + w_t^w$$

$$w_t^u \leftarrow \gamma w_{t-1}^u + w_t^r + w_t^w$$

$$w_t^w \leftarrow \sigma(\alpha) w_{t-1}^r + (1 - \sigma(\alpha)) w_{t-1}^{lu}$$

$$M_t(i) \leftarrow M_{t-1}^i + w_t^w(i) k_t$$

Differentiable Neural Computers

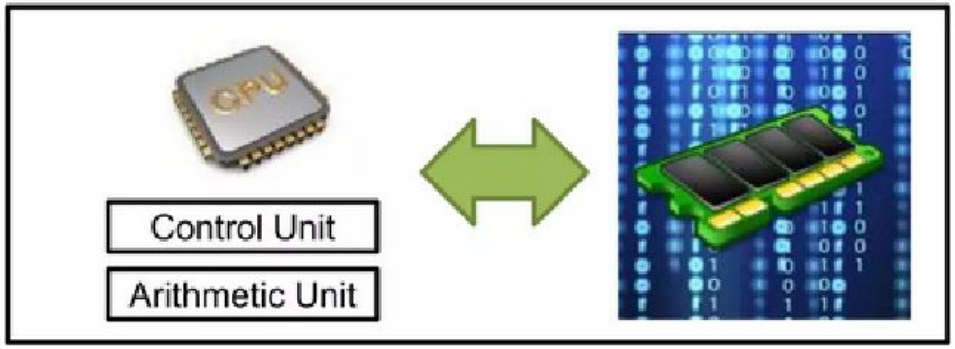
- Limitations of NTM
  - have no way to ensure that no interference or overlap between written data would occur.
  - The inability to free and reuse memory locations
  - If the write head jumped to another place in the memory while writing some consecutive data, a read head won't be able to recover the temporal link between the data written before and after the jump: this constitutes the third limitation of NTMs.

Differentiable Neural Computers

Concept of DNC

- Modern computers separate computation and external memory (RAM)
  - Computation is performed by a CPU
  - CPU can use an addressable memory through virtual memory
- Benefits
  - Use of extensible storage
  - Treat the contents of memory as variable

  → Algorithm generality: to perform same procedure on one datum or another

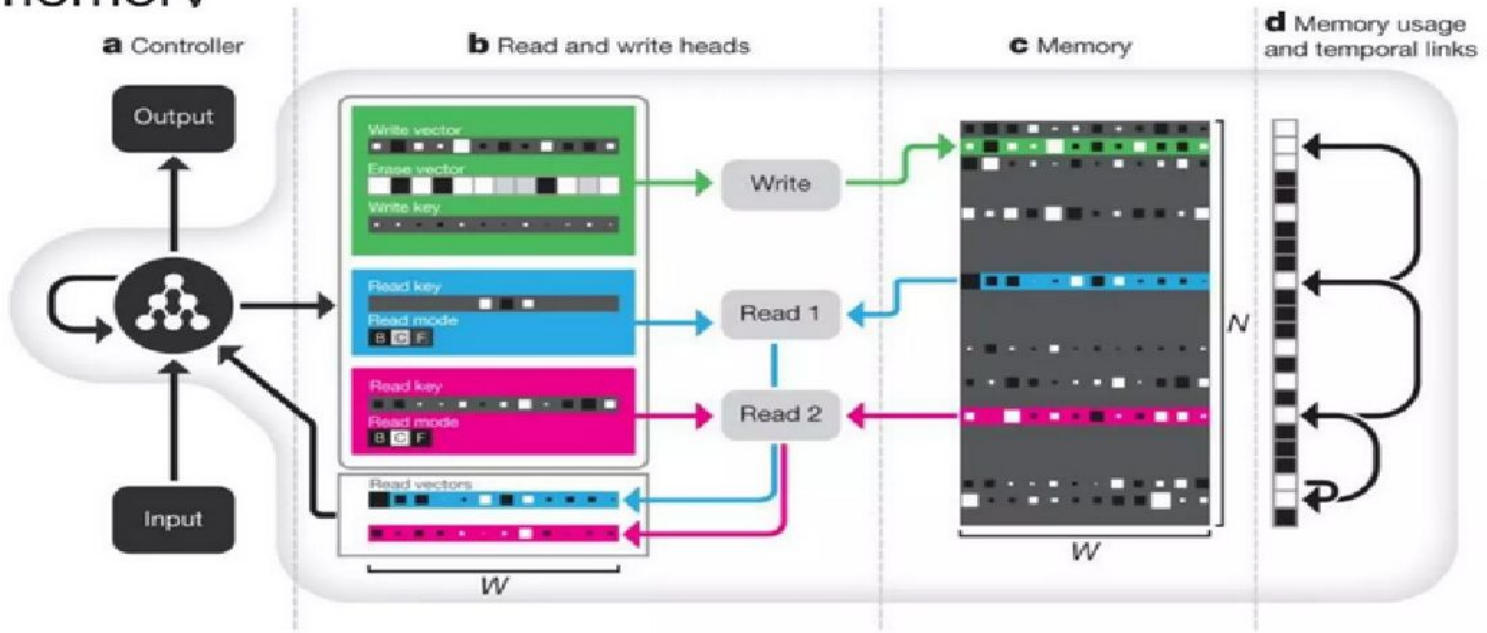


Differentiable Neural Computers

Concept of DNC

- DNC consists of 3 parts;

  (i) Controller, *(ii)* Memory interface, *(iii)* External memory



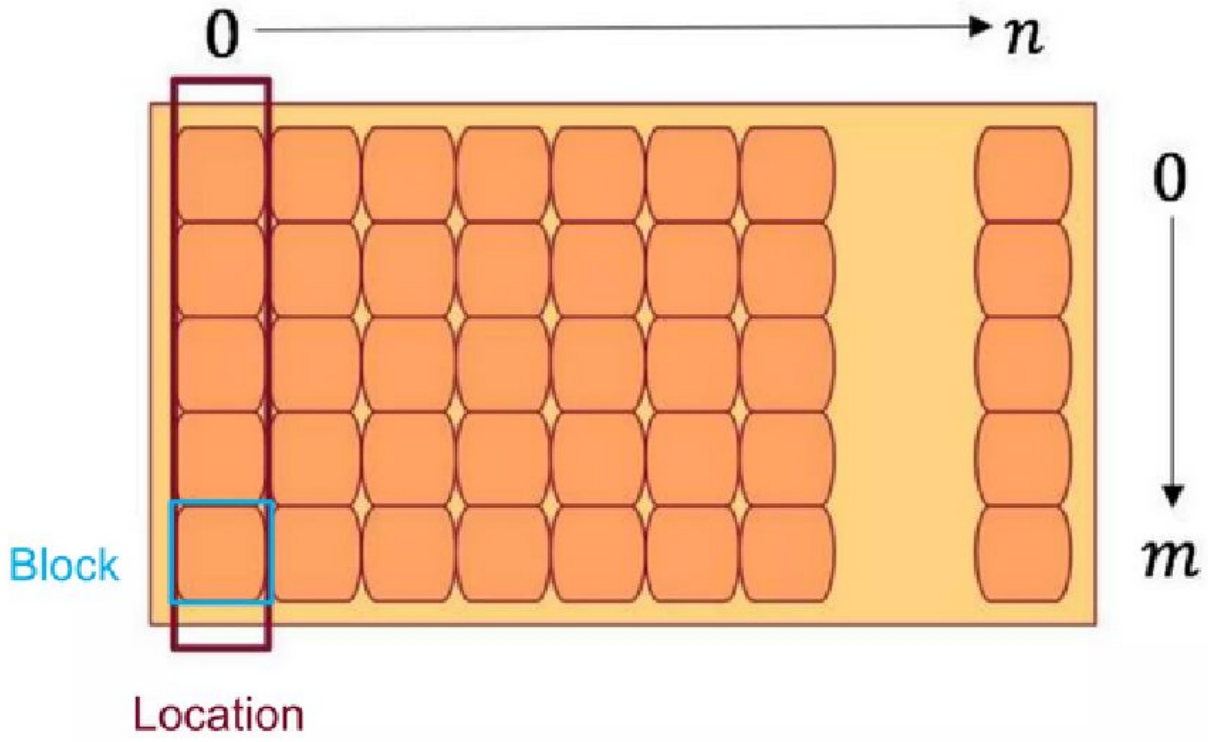
Overall Architecture of DNC

Illustration of the DNC architecture. The neural network controller receives external inputs and, based on these, interacts with the memory using read and write operations known as 'heads'. To help the controller navigate the memory, DNC stores 'temporal links' to keep track of the order things were written in, and records the current 'usage' level of each memory location.

Differentiable Neural Computers

Memory Structure

- M_t is $N \times M$ matrix of memory at time t.
- N locations in memory, each location has M blocks
- Block is minimum unit for reading and writing data



Differentiable Neural Computers

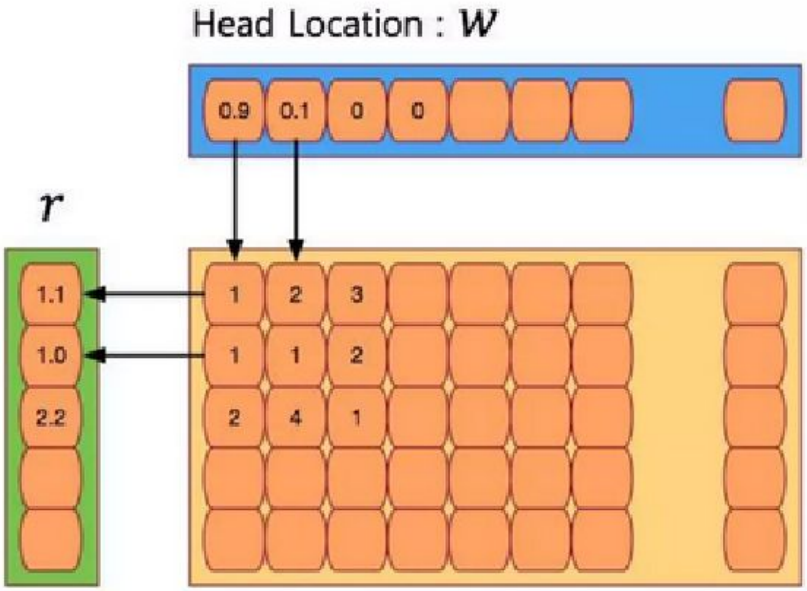
Read Mechanism

- Selecting locations for reading depends on weightings
- Read vector: output of reading

$$r_t^i = M_t^\top w_t^{r,i} \in \mathbb{R}^W$$

  - Reading weightings: $w_t^r \ \{w_t^{r,1}, \ldots, w_t^{r,R} \in \Delta_N\}$

    where $\Delta_N = \{\alpha \in \mathbb{R}^N : \alpha_i \in [0,1], \sum_{i=1}^N \alpha_i \leq 1\}$



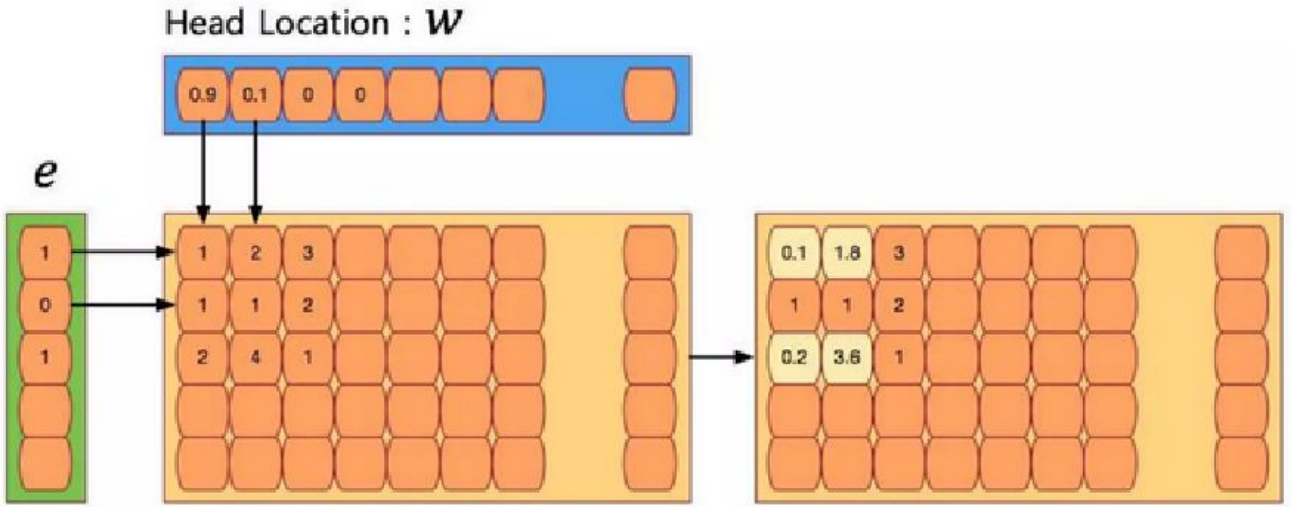
Differentiable Neural Computers

Write Mechanism

- Writing involves *(i) erasing* and *(ii) adding* steps.
  - Taking inspiration from the *forget* and *input* gates in LSTM
- *Erasing step*

$$\widetilde{M}_t = M_{t-1} \circ (\boldsymbol{E} - \boldsymbol{w}_t^{\boldsymbol{w}} \boldsymbol{e}_t^{\mathrm{T}})$$

  - Erase vector: $e_t \in [0,1]^W$
  - Writing vector: $w_t^w \in \Delta_N$



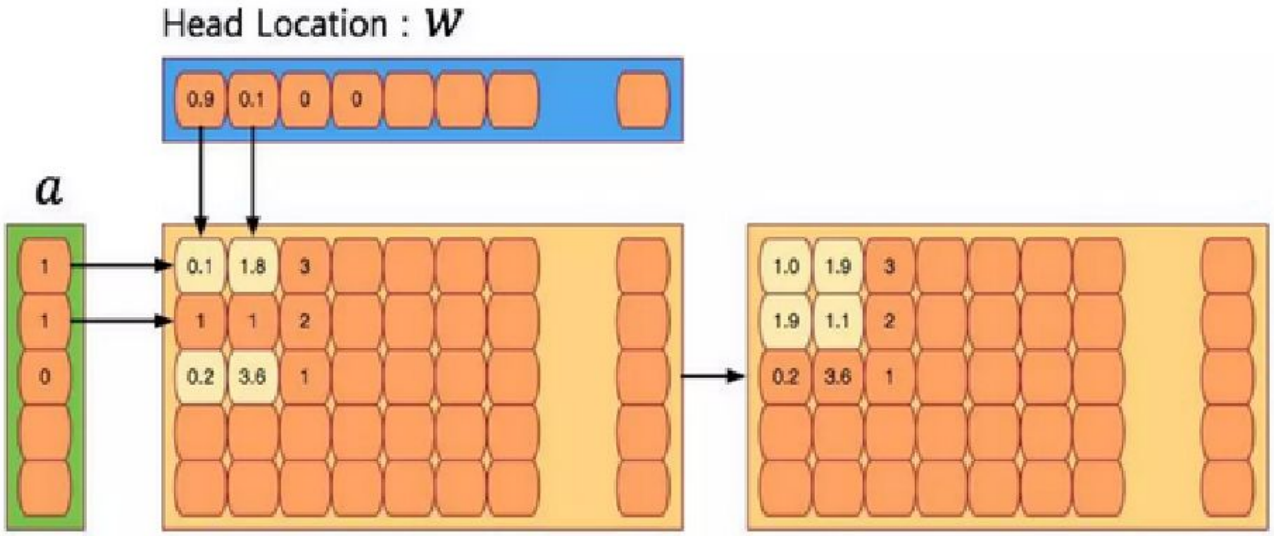
Differentiable Neural Computers

Write Mechanism

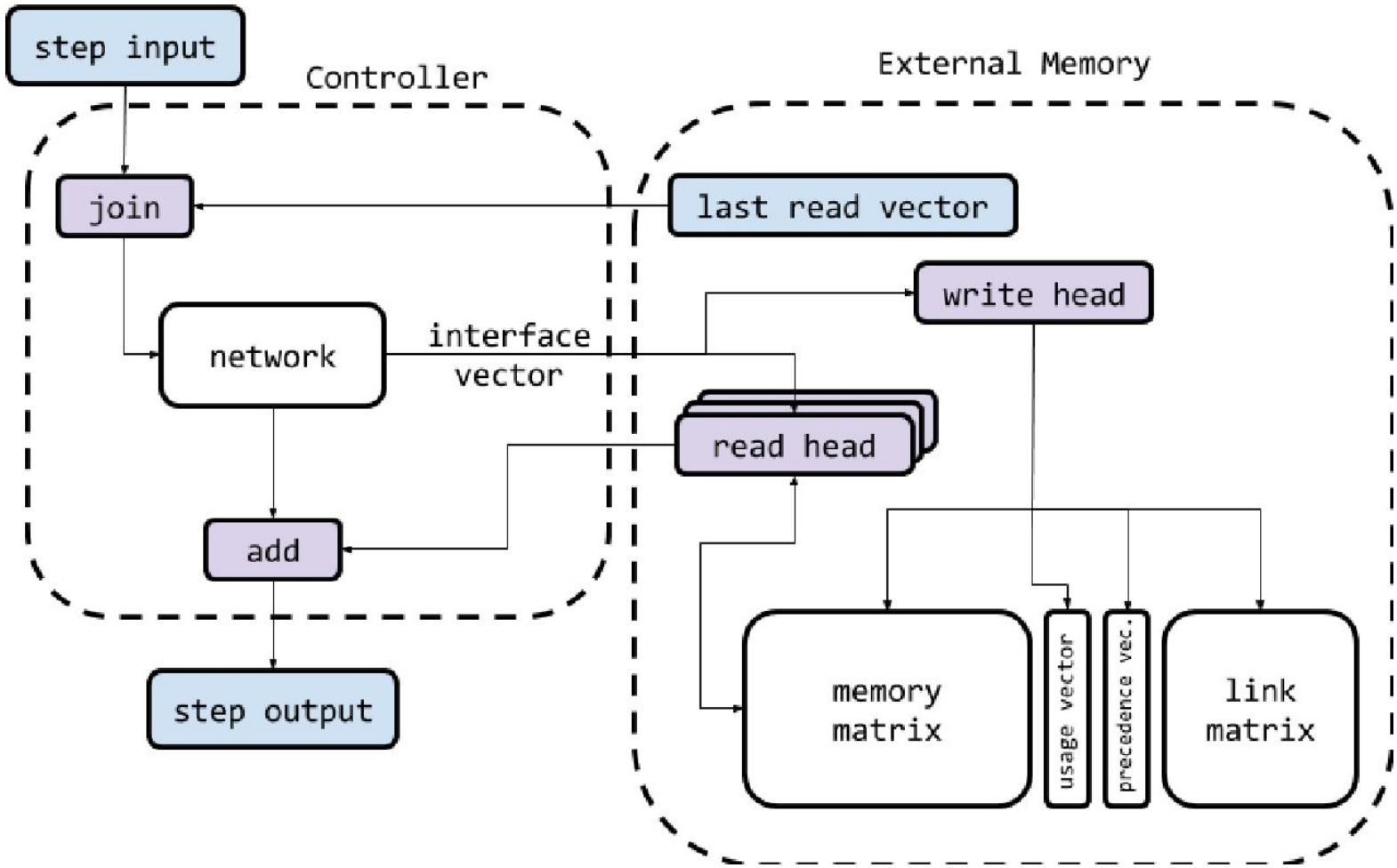
- Adding step
  - Add vector ($v_t \in \mathrm{R}^W$) is added to the memory after erase step

$$\mathrm{M_t} = M_{t-1} \circ \left(E - \boldsymbol{w}_t^w \boldsymbol{e}_t^{\mathrm{T}}\right) + \boldsymbol{w}_t^w \boldsymbol{v}_t^{\mathrm{T}}$$

  - Add vector (data) and weightings are given by controller



Differentiable Neural Computers



https://www.deepmind.com/blog/differentiable-neural-computers

Differentiable Neural Computers

- The external memory then takes in the interface vector, performs the necessary writing through a single write head, then reads *R* new vectors from the memory.

- It returns the newly read vectors to the controller to be added with the pre-output vector, producing the final output vector of size *Y*.

• Interference-Free Writing in DNCs

- The first limitation we discussed of NTMs was their inability to ensure an interference-free writing behavior.

- An intuitive way to address this issue is to design the architecture to focus strongly on a single, free memory location and not wait for NTM to learn to do so.

- In order to keep track of which locations are free and which are busy, we need to introduce a new data structure that can hold this kind of information.

- We'll call it the *usage vector.*

- The usage vector u_*t* is a vector of size *N* where each element holds a value between 0 and 1 that represents how much the corresponding memory location is used; with 0 indicating a completely free location and 1 indicating a completely used one.

Differentiable Neural Computers

- The usage vector initially contains zeros u_0 = **0** and gets updated with the usage information across the steps.
- Using this information, it's clear that the location to which the weights should attend most strongly to is the one with the least usage value.
- To obtain such weighting, we need first to sort the usage vector and obtain the list of location indices in ascending order of the usage; we call such a list a *free list* and denote it by ϕ_t.
- Using that free list, we can construct an intermediate weighting called the *allocation weighting a_t* that would determine which memory location should be allocated for new data.

$$a_t[\phi_t[j]] = (1 - u_t[\phi_t[j]]) \prod_{i=1}^{j-1} u_t[\phi_t[i]] \quad \text{where } j \in 1, \cdots, N$$

$1 - u_t[\phi_t[j]]$ makes the location weight proportional to how free it is.

$\prod_{i=1}^{j-1} u_t[\phi_t[j]]$ least used location having the largest weight, while the most used one gets the smallest weight.

Differentiable Neural Computers

- With the allocation weighting *a_t* and lookup weighting *c_t^w,* we can now construct our final write weighting:

$$\mathrm{w}_t^w = g_t^w[g_t^a \mathrm{a}_t + (1 - g_t^a)\mathrm{c}_t^w]$$

where g_t^w, g_t^a are values between 0 and 1 called the write and allocation gates, which we also get from the controller through the interface vector. These gates control the writing operation with g_t^w determining if any writing is going to happen in the first place, and g_t^a specifying whether we'll write to a new location using the allocation weighting or modify an existing value specified by the lookup weighting.

DNC Memory Reuse

- What if while we calculate the allocation weighting we find that all locations are used, or in other words u_*t* = **1**?
- This means that the allocation weightings will turn out all zeros and no new data can be allocated to memory.
- This raises the need for the ability to free and reuse the memory.
- In order to know which locations can be freed and which cannot, we construct a *retention vector* ψ_t of size N that specifies how much of each location should be retained and not get freed.
- Each element of this vector takes a value between 0 and 1, with 0 indicating that the corresponding location can be freed and 1 indicating that it should be retained.
- This vector is calculated using:

$$\psi_t = \prod_{i=1}^{R} \left(\mathbf{1} - f_t^i \mathrm{w}_{t-1}^{r,i} \right)$$

DNC Memory Reuse

- This equation is basically saying that the degree to which a memory location should be freed is proportional to how much is read from it in the last time steps by the various read heads (represented by the values of the read weightings w_*t* -1^*r*, *i*).

- However, continuously freeing a memory location once its data is read is not generally preferable as we might still need the data afterward.

- We let the controller decide when to free and when to retain a location after reading by emitting a set of *R* free gates *f_t^1*, ···, *f_t^R* that have a value between 0 and 1.

- This determines how much freeing should be done based on the fact that the location was just read from.

- The controller will then learn how to use these gates to achieve the behavior it desires.

- Once the retention vector is obtained, we can use it to update the usage vector to reflect any freeing or retention made via:

$$u_t = \left(u_{t-1} + w^w_{t-1} - u_{t-1} \circ w^w_{t-1}\right) \circ \psi_t$$

DNC Memory Reuse

- a location will be used if it has been retained (its value in $\psi\_t \approx 1$) and either it's already in use or has just been written to (indicated by its value in u_$t-1$ + w_$t-1$^w).
- Subtracting the element-wise product u_$t-1 \circ$ w_$t-1$^w brings the whole expression back between 0 and 1 to be a valid usage value in case the addition between the previous usage got the write weighting past 1.

$$u_t = \left(u_{t-1} + w^w_{t-1} - u_{t-1} \circ w^w_{t-1}\right) \circ \psi_t$$

Temporal Linking of DNC Writes

- With the dynamic memory management mechanisms that DNCs use, each time a memory location is requested for allocation, we're going to get the most unused location, and there'll be no positional relation between that location and the location of the previous write. (Ex: Tirumala Tirupati Room Allocation)

- With this type of memory access, NTM's way of preserving temporal relation with contiguity is not suitable.

- We'll need to keep an explicit record of the order of the written data.

- This explicit recording is achieved in DNCs via two additional data structures alongside the memory matrix and the usage vector.

- The first is called a *precedence vector* p_*t*, an *N*-sized vector considered to be a probability distribution over the memory locations, with each value indicating how likely the corresponding location was the last one written to.

- The precedence is initially set to zero p_0 = **0** and gets updated in the following steps via:

$$p_t = \left(1 - \sum_{i=1}^{N} w_t^w[i]\right) p_{t-1} + w_t^w$$

Temporal Linking of DNC Writes

- Updating is done by first resetting the previous values of the precedence with a reset factor that is proportionate to how much writing was just made to the memory (indicated by the summation of the write weighting's components).

- Then the value of write weighting is added to the reset value so that a location with a large write weighting (that is the most recent location written to) would also get a large value in the precedence vector.

The DNC Controller Network

- The controller's operation is simple: in its heart there's a neural network (recurrent or feed-forward) that takes in the input step along with the read-vectors from the last step and outputs a vector whose size depends on the architecture we chose for the network.

- Let's denote that vector by $\mathcal{N}(\chi\_t)$, where $\mathcal{N}$ denotes whatever function is computed by the neural network, and χ_t denotes the concatenation of the input step and the last read vectors $\chi\_t = x\_t; r\_t - 1\hat{}1 ; \cdots; r\_t - 1 \hat{} R$.

- From that vector emitted from the neural network, we need two pieces of information. The first one is the interface vector ζ_t. As we saw, the interface vector holds all the information for the memory to carry out its operation.

$$\zeta_t = [\underbrace{k_t^{r,1}; \ldots; k_t^{r,R}}_{\text{each of size } W}; \overbrace{\beta_t^{r,1}; \ldots; \beta_t^{r,R}}^{\text{each of size 1}}; k_t^w; \beta_t^w; \underbrace{e_t; v_t}_{\text{size } W}; \overbrace{f_t^1; \ldots; f_t^R; g_t^a; g_t^w}^{\text{each of size 1}}; \underbrace{\pi_t^1; \ldots; \pi_t^R}_{\text{each of size 3}}]$$

The interface vector decomposed to its individual components

Thank you